PUCE

Programación Móvil

Semana 2

Kotlin Moderno — Fundamentos y Seguridad de Tipos.

TEMAS

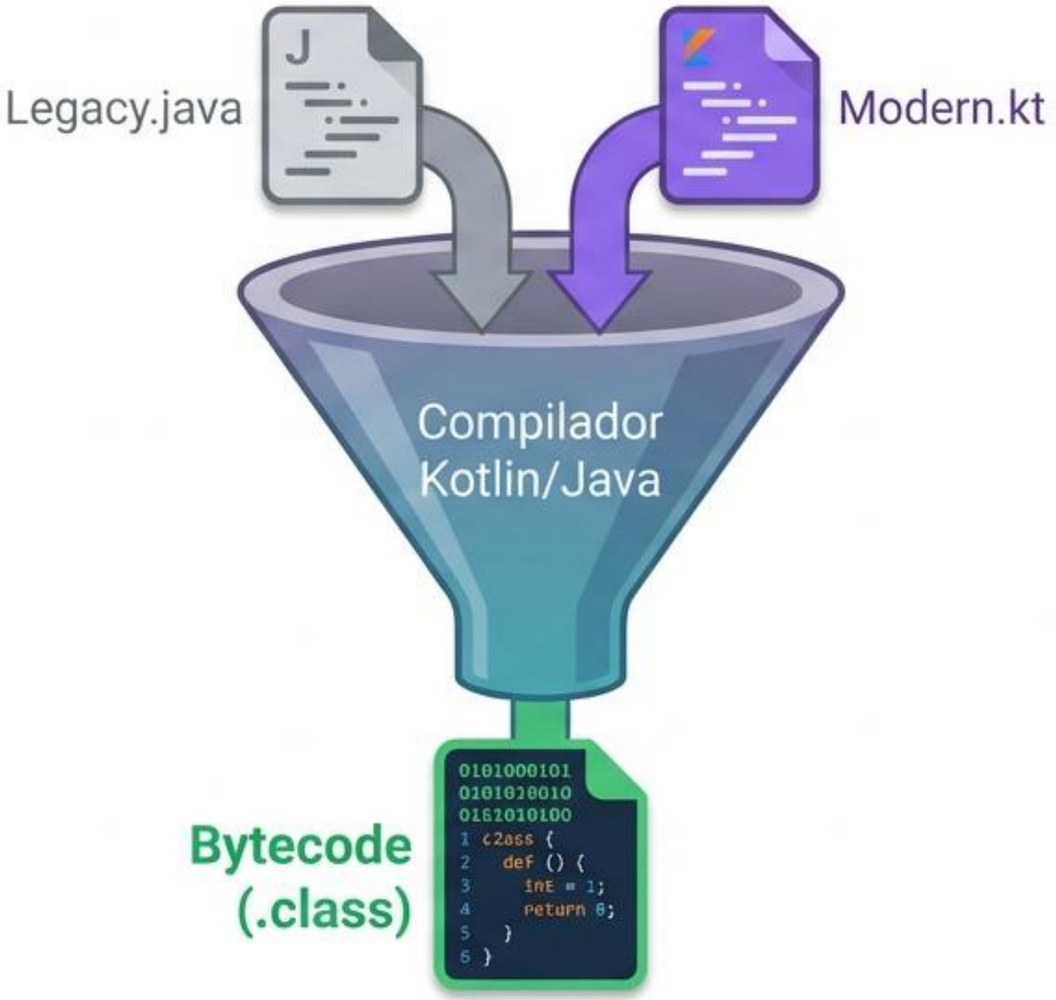
1. Por qué Kotlin: El Sucesor de Java.
2. Sistema de Tipos e Inferencias.
3. Control de Flujo: when como expresión potente.
4. Convenciones de Estilo y Buenas Prácticas.
5. Comparativa iOS: Swift Optionals vs Kotlin Null Safety

OBJETIVOS

1. Evaluar las ventajas de Kotlin frente a Java.
2. Aplicar el sistema de tipos de Kotlin.
3. Implementar estructuras de control de flujo avanzadas.
4. Realizar análisis comparativo con el sistema Optionals de Swift.

Kotlin no busca reemplazar la JVM, sino evolucionar la ingeniería sobre ella.

- **Evolución, no reemplazo**: Traduce al mismo bytecode (.class).
- **Productividad**: Reduce la deuda técnica industrial.
- **Soporte Oficial**: Estándar de Google para Android.



Interoperabilidad Sin Costo: La Filosofía de Coexistencia

- **Coexistencia Total:** Kotlin no requiere reescribir la aplicación. Archivos **.java** y **.kt** residen en el mismo proyecto.
- **Sin Puentes Costosos:** No hay serialización ni 'bridges' intermedios. La llamada entre lenguajes es directa.
- **Migración Gradual:** Permite mantener lógica de negocio heredada (Legacy) mientras se desarrollan nuevas funcionalidades en Kotlin.



Ejemplo de Interoperabilidad Java + Kotlin

Java

```java
public class UsuarioLegacy {
    private String nombre;
    public UsuarioLegacy(String nombre) { this.nombre = nombre; }
    public String getNombre() { return nombre; }
}
```

Kotlin

```kotlin
val u = UsuarioLegacy("Ana")
println(u.nombre)   // Kotlin convierte getNombre() en propiedad
```

- Kotlin y Java conviven en el mismo proyecto
- Kotlin puede llamar a Java directamente
- Java puede llamar a Kotlin sin cambios
- No hay pérdida de rendimiento
- Permite migraciones graduales
- Reduce costos y riesgos en proyectos reales

El Reto de la Interoperabilidad: Platform Types

Java (Legacy)

```java
public String getPayload() {
  return value;
}
```

Kotlin (Consumidor)

```kotlin
val data = javaObject.getPayload()
// Tipo inferido: String!
```

- **El Dilema del Compilador:** Cuando Java no define explícitamente @Nullable o @NotNull, Kotlin no puede garantizar la seguridad.
- **Platform Types (Tipos de Plataforma):** Kotlin asigna un tipo flexible (ej. String!) delegando la responsabilidad de validación al ingeniero.
- **Riesgo:** Tratar un Platform Type como no-nulo sin validación es la causa principal de NullPointerException en entornos mixtos.

Ejemplo de Null-Point Exception en la Interoperatividad

Java

```java
public class LegacyData {
    public String[] getNombres() {
        return new String[3]; // [null, null, null]
    }
}
```

Kotlin

```kotlin
fun main() {
    val legacy = LegacyData()
    val nombres = legacy.getNombres()   // Tipo inferido: Array<String!>

    println(nombres[0].length)          // 💥 NPE desde Kotlin
}
```

¿Cómo debería manejarse correctamente?

✔ Opción 1: Validar manualmente

Kotlin

```kotlin
val primero = nombres[0]
if (primero != null) {
    println(primero.length)
}
```

✔ Opción 2: Usar el operador seguro `?.`

Kotlin

```kotlin
println(nombres[0]?.length)
```

Si es null → imprime `null`

Si no → imprime la longitud

✔ Opción 3: Corregir el código Java

(La mejor solución en proyectos reales)

Java

```java
@NotNull
public String[] getNombres() { ... }
```

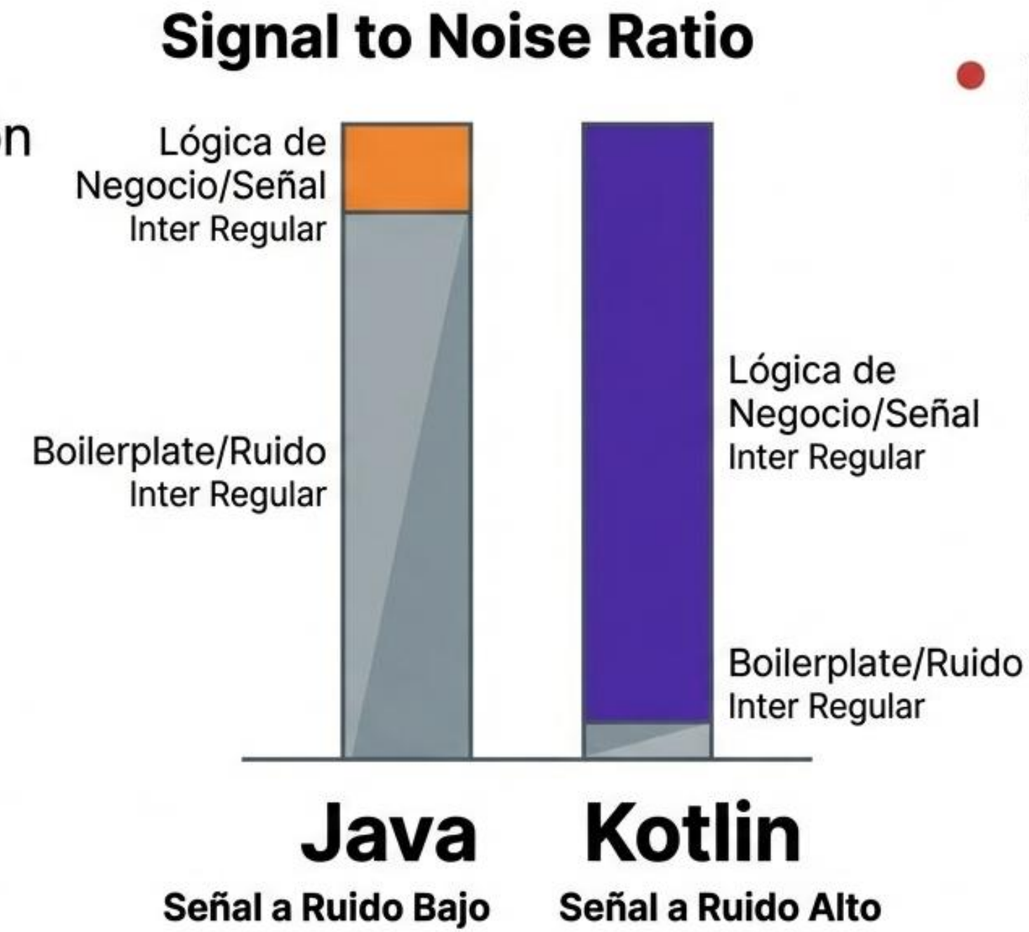
O si puede devolver null:

Java

```java
@Nullable
public String[] getNombres() { ... }
```

Concisión: Elevando el Nivel de Abstracción

- **Más que Ahorrar Teclas:** La concisión reduce la carga cognitiva.
- **Eliminación de 'Ruido':** El compilador genera el código repetitivo.



- **Mantenibilidad:** Menos código = Menos bugs.

Menos código, mayor abstracción.

Java (Verbosidad Estructural)

```
public class User {
    private String id;
    private String name;

    public User(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public boolean equals(Object o) { ... }

    @Override
    public int hashCode() { ... }

    @Override
    public String toString() { ... }

}
```

OBSOLETO

Kotlin (Concisión Declarativa)

```
data class User(val id: String,
    val name: String)
```

Automatización del Compilador: Genera automáticamente **equals()**, **hashCode()**, **toString()**, y **copy()**. Resultado: Código limpio y sin deuda técnica.

```
// Nota: Kotlin delega la generación
    de boilerplate al compilador
```

Java

```java
import java.util.Objects;

class Persona {
    String nombre;
    int edad;

    Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Persona)) return false;
        Persona persona = (Persona) o;
        return edad == persona.edad &&
               Objects.equals(nombre, persona.nombre);
    }
    @Override
    public int hashCode() {
        return Objects.hash(nombre, edad);
    }

    @Override
    public String toString() {
        return "Persona(nombre=" + nombre + ", edad=" + edad + ")";
    }
}
public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona("Ana", 20);
        Persona p2 = new Persona("Ana", 20);

        System.out.println("p1.equals(p2): " + p1.equals(p2));

        java.util.HashSet<Persona> set = new java.util.HashSet<>();
        set.add(p1);
        System.out.println("¿El set contiene a p2?: " + set.contains(p2));

        System.out.println("toString(): " + p1);
    }
}
```

Kotlin

```kotlin
data class Persona(val nombre: String, val edad: Int)

fun main() {
    val p1 = Persona("Ana", 20)
    val p2 = Persona("Ana", 20)

    println("p1 == p2: ${p1 == p2}")        // true
    println("p1.equals(p2): ${p1.equals(p2)}") // true

    println("toString(): $p1")              // Persona(nombre=Ana, edad=20)

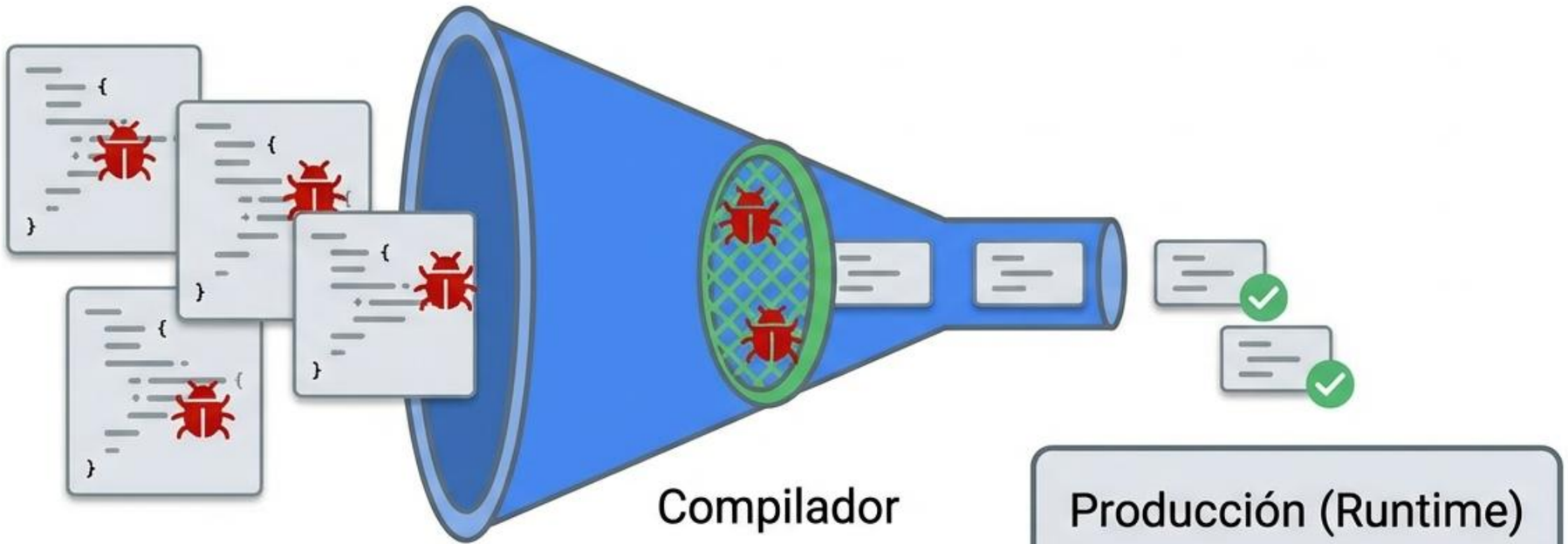
    val set = hashSetOf(p1)
    println("¿El set contiene a p2?: ${set.contains(p2)}") // true
}
```

Con `data class`, Kotlin:

- Compara contenido correctamente
- Funciona bien con colecciones basadas en hash
- Produce un `toString()` legible
- Evita errores silenciosos
- Reduce 50 líneas de Java a 1 línea

Seguridad Estática (Compile-time Safety)

El compilador actúa como el primer filtro de calidad, atrapando errores antes de que lleguen al usuario.



"Si compila, es mucho más probable que funcione correctamente."

Fallar Rápido: Tiempo de Compilación vs. Ejecución

Tiempo de Compilación (IDE)

Tiempo de Ejecución (Usuario)

```
import Kotlin

func mainlisten() {
    var = getaspaint()
    ...
}
```

Kotlin traslada la detección de errores a la fase de desarrollo.



Error Detectado:
Seguro y Eficiente

Java: Costoso y Crítico

Declaración Explícita

```
val nombre: String = 'Kotlin'
```

Inferencia de Tipo

```
val nombre = 'Kotlin'
```

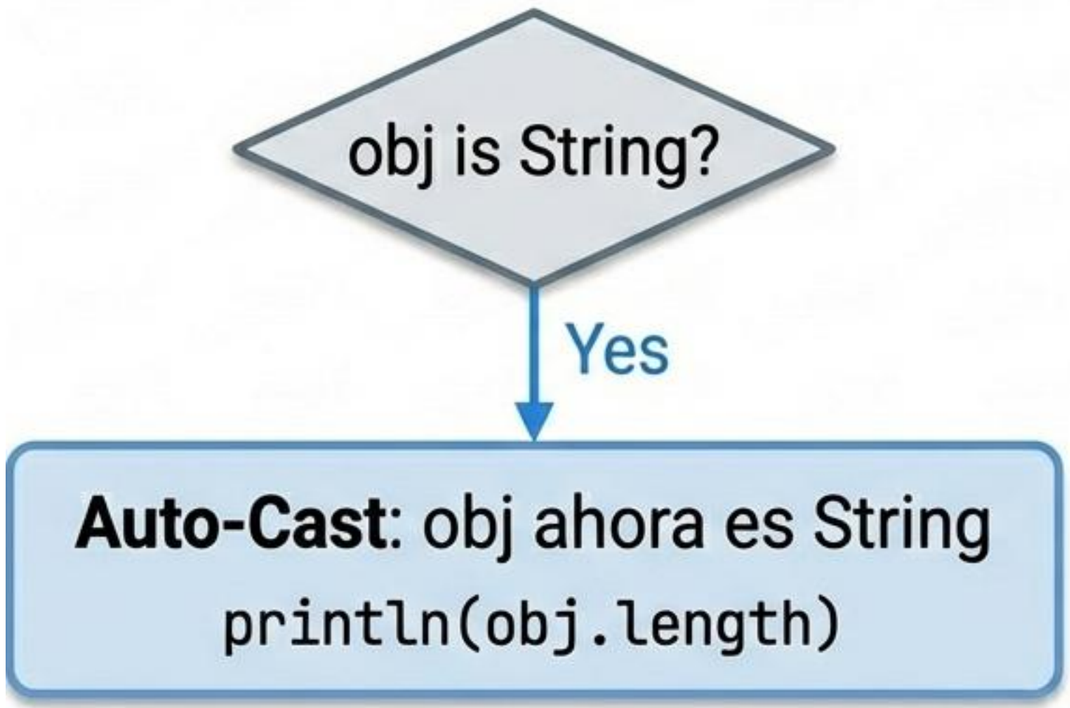
El compilador deduce que esto es un String

- **Tipado Estático Fuerte**: La variable tiene un tipo fijo definido en compilación.
- **Deducción Automática**: Análisis del valor asignado.
- **Seguridad**: Una vez inferido, el tipo no puede cambiar.

Elimina redundancia visual sin sacrificar seguridad.

Smart Casts (Moldeado Inteligente)

El compilador rastrea el flujo de control y promociona los tipos automáticamente.



Auto-Cast

```kotlin
val obj: Any = "Hola"

if (obj is String) {
    // No se requiere (String) obj
    println(obj.length)
}
```

```kotlin
val nombre: String? = "Ana"

if (nombre != null) {
    println(nombre.length)  // Smart cast a String
}
```

Seguridad por Diseño: Inmutabilidad

val (Valor)



Referencia de solo lectura. Inmutable. Thread-Safe.

var (Variable)



Referencia mutable. Estado cambiante. Requiere gestión manual.

Regla de Oro

Filosofía "Inmutabilidad Primero": Siempre declarar con **val**. Cambiar a **var** solo si es estrictamente necesario.

🧠 ¿Qué significa realmente val?

val = **inmutable**

- Solo se asigna **una vez**.
- No puede cambiar después.
- Es como final en Java.

Ejemplo:

```
val edad = 20
edad = 21 // ❌ Error: Val cannot be reassigned
```

🔄 ¿Y si la edad del usuario cambia?

Entonces no debe usar val, sino var.

```
var edad = 20
edad = 21   // ✔ Correcto
```

🧠 ¿Qué pasa si quiero usar val pero necesito actualizar “algo”?

```
data class Usuario(var edad: Int)

fun main() {
    val usuario = Usuario(20)

    usuario.edad = 21   // ✔ permitido
    usuario = Usuario(30) // ❌ NO permitido,
                          porque usuario es val
}
```

- La referencia usuario no cambia, pero su contenido sí.
- Esto es útil cuando quiere proteger la referencia, pero no el estado interno.

Comparativa Técnica y Recomendaciones de Uso

Característica	val (Inmutable)	var (Mutable)
Naturaleza	Referencia de solo lectura	Variable de lectura y escritura
Reasignación	Prohibida tras la inicialización	Permitida en cualquier momento
Seguridad de Hilos	Intrínsecamente segura para concurrencia	Requiere gestión manual de estados
Optimización	El compilador K2 puede aplicar mayores optimizaciones	Menos margen para optimización de flujo
Escenario Ideal	Constantes, identificadores, configuraciones, referencias a objetos de UI	Contadores, entradas de usuario, estados efímeros

Consecuencias técnicas y arquitectónicas de la Inmutabilidad

- La programación se basa en **transformaciones de datos seguras** evitando manipulaciones directas del estado.
- En Jetpack Compose, se usa el paradigma de **estabilidad de estado**: crear copias inmutables en lugar de modificar estados mutables.
- Beneficios en aplicaciones móviles:
  - **Debugging simplificado:** valores constantes facilitan el rastreo y evitar errores.
  - **Optimización de renderizado:** evita recomposiciones innecesarias al garantizar inmutabilidad.
  - **Seguridad en concurrencia:** objetos inmutables son seguros para uso en múltiples hilos, previniendo condiciones de carrera.

Kotlin y la Nulidad: El "Error del Billón de Dólares"

🔥 Problema histórico

- Tony Hoare introdujo null en 1965 y luego lo llamó su "**error del billón de dólares**" por los millones de fallos y NPEs que causó.
- En Java, **cualquier referencia puede ser null**, lo que obliga a escribir código defensivo:

```
if (obj != null) {
   obj.doSomething();
}
```

Kotlin y la Nulidad: El "Error del Billón de Dólares"

🔥 **En Kotlin: la nulidad es parte del sistema de tipos**

- Por defecto, las variables NO pueden ser nulas.
- Kotlin divide los tipos en:
  - **No nulos**: String
  - **Nulos**: String?

```
val nombre: String = "Ana"    // ✔ Seguro
val token: String? = null     // ✔ Permitido, pero controlado
```

Manejo seguro de null en Kotlin

🔥 Seguridad garantizada por el compilador

- Si un tipo es no nulo, **jamás** puede contener `null`.
- Si un tipo es `nullable`, Kotlin exige comprobación previa:

```
val token: String? = null
println(token.length)    // ❌ Error de compilación
```

✔️ Verificación explícita

```
if (token != null) {
   println(token.length)    // Smart cast a String
}
```

✔️ Operador seguro ?.

```
println(token?.length)
```

Manejo seguro con el Operador Elvis (?:)

Transformando la incertidumbre en control.




```
val estado = respuestaAPI ?: "Sin conexión"
```

Valor de respaldo

Operador de Aserción No Nula (!!): El Botón de Pánico

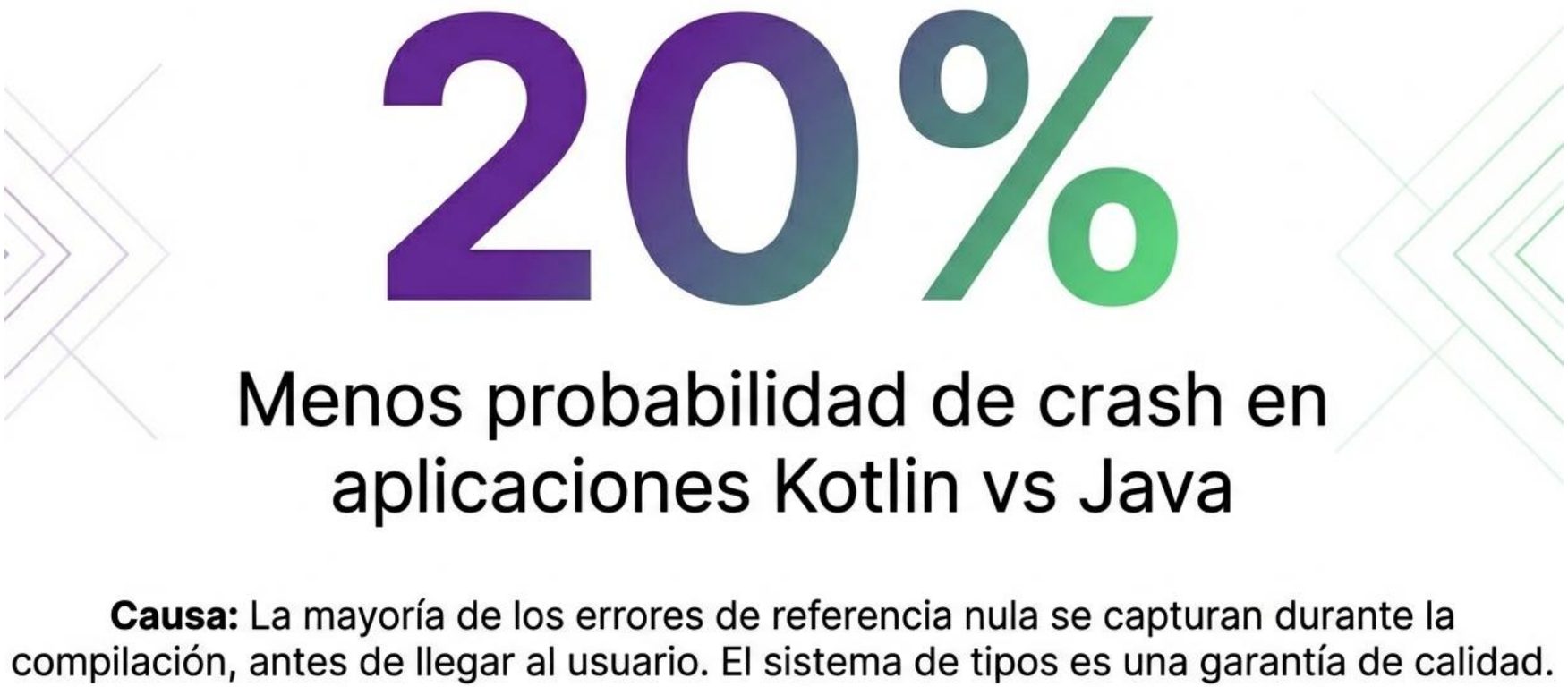
```
val len = texto!!.length
```





- **Función:** Fuerza a una variable anulable a ser tratada como no-nula.
- **Riesgo Crítico:** Lanza NullPointerException inmediato si el valor es null.
- **Mejor Práctica:** Considerado un 'Code Smell'. Evitar en producción.

El impacto de la Seguridad de Tipos



'when' como Expresión

`when` devuelve un valor. Obliga a la **exhaustividad** (cubrir todos los casos).



Switch en Java

```java
String color;
switch (score) {
    case 100:
    case 99:
    case 98:
    case 97:
    case 96:
    case 95:
    case 94:
    case 93:
    case 92:
    case 91:
    case 90:
        color = "Dorado";
        break;
    default:
        color = "Gris";
}
```

Control de Flujo: 'when' vs 'switch'

Comparativa de sentencias de selección múltiple.

Java (switch)

```
switch (x) {
   case 1:
      System.out.println("x is 1");
      break;
   case 2:
      System.out.println("x is 2");
      break;
   default:
      System.out.println("x is neither");
}
```

Requiere 'break' para evitar caídas.

Kotlin (when)

```
when (x) {
    1 -> print("x is 1")
    2 -> print("x is 2")
    else -> print("x is neither")
}
```

Sin 'break'. Flecha `->` limpia. Expresivo.

- Expresión vs. Sentencia: **when** devuelve un valor asignable.
- Exhaustividad: Obliga a cubrir todos los casos posibles.
- Diseño Limpio: Elimina el error del "fall-through" (olvidar el break).

Pattern Matching avanzado con 'when'

```
when (x) {
    0, 1 -> print('Binario')
    in 10..20 -> print('Rango')
    is String -> print('Es texto')
    else -> print('Otro')
}
```

Rangos

Verificación de Tipo + Smart Cast

- **Versatilidad:** Evalúa rangos, tipos y condiciones lógicas, no solo constantes.

Declaración Estática Imperativa vs. Expresión Dinámica y Exhaustiva

Java Clásico

```java
public String getFinancialPeriodType(int reportingQuarter) {
    String typeOfPeriod; // Variable mutada a posteriori
    switch (reportingQuarter) {
        case 1:
        case 4:
            typeOfPeriod = "Semestre de Alta Demanda";
            break; // Crítico: Si el ingeniero olvida esta línea, el flujo cae en cascada y sobreescribe datos.
        case 2:
        case 3:
            typeOfPeriod = "Semestre de Estabilización Fiscal";
            break;
        default:
            throw new IllegalArgumentException("Trimestre contable fuera de los límites del calendario.");
    }
    return typeOfPeriod;
}
```

Kotlin Moderno

```kotlin
fun getFinancialPeriodType(reportingQuarter: Int): String =
when (reportingQuarter) {
    1, 4 -> "Semestre de Alta Demanda" // Ramificación múltiple concisa separada por coma
    in 2..3 -> "Semestre de Estabilización Fiscal" // Soporte nativo y expresivo para evaluación continua de rangos numéricos
    else -> throw IllegalArgumentException("Trimestre contable fuera de los límites del calendario.")
}
```

Nomenclatura Profesional

Kotlin, apoyado por Google y JetBrains, sigue una guía de estilo estricta:

- **Clases y Objetos:** Utilizar PascalCase (ejemplo: UserAccount, LoginFragment).
- **Funciones y Propiedades:** Utilizar camelCase (ejemplo: calculateFinalPrice, isUserLoggedIn).
- **Constantes:** Variables de nivel superior o dentro de objetos compañeros marcadas con const val deben usar SCREAMING_SNAKE_CASE (ejemplo: BASE_URL, MAX_RETRY_COUNT).

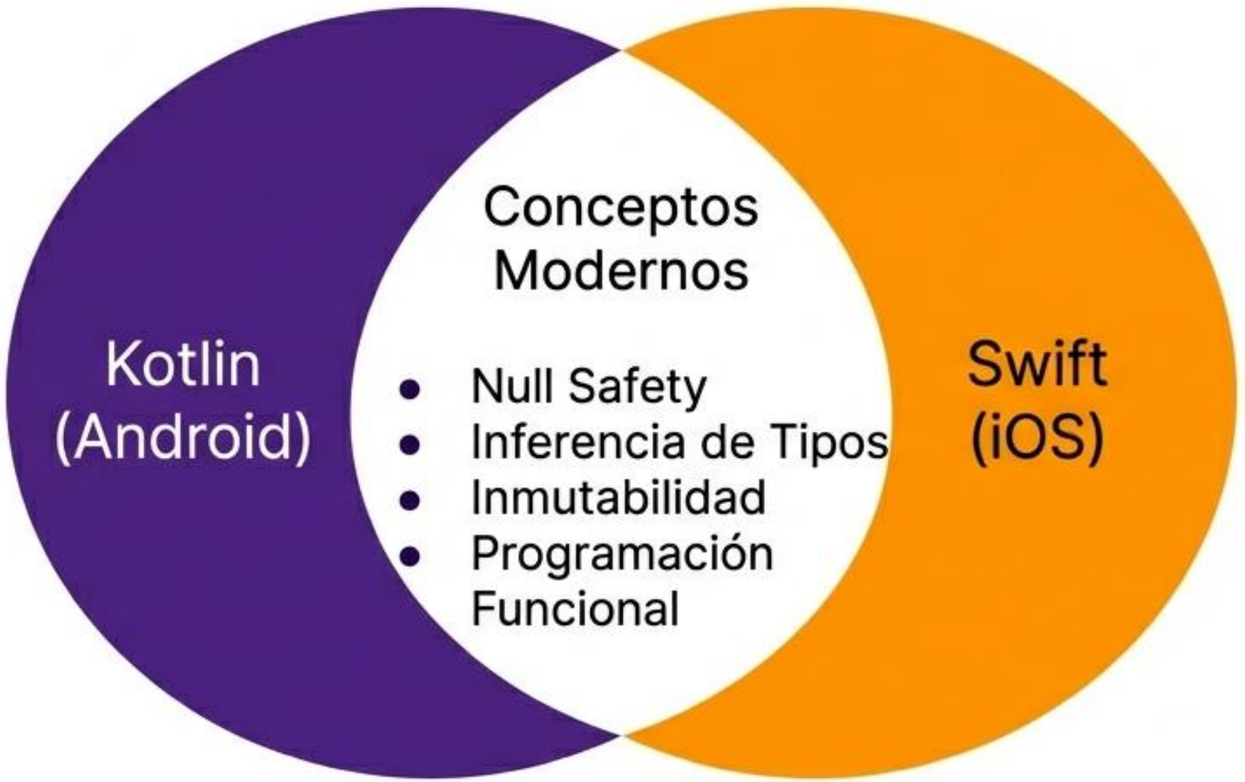
Estructura de Archivos y Espacios en Blanco

El orden de los elementos en un archivo **.kt** es vital para la navegación rápida:

- Encabezados de licencia.
- Sentencia de paquete.
- Importaciones (agrupadas y sin usar comodines *).
- Declaraciones de clases o funciones.

Para mayores detalles en cuanto al manejo de Estilo de Kotlin consultar la Guía de estilo de Kotlin oficial.

Convergencia Móvil Kotlin y Swift



Lenguajes Gemelos: Ambos evolucionaron para resolver los problemas de C/Java. **Sintaxis compartida:** `String?` / `String?`, `null` / `nil`.

Comparativa iOS: Paralelismos con Swift

Kotlin y Swift comparten filosofías de seguridad modernas.

Feature	Kotlin	Swift
Tipo Anulable	`String?`	`String?`
Valor por Defecto	`?:` (Elvis)	`??` (Nil-Coalescing)
Inmutabilidad	`val`	`let`
Llamada Segura	`user?.name`	`user?.name`

Aprender Kotlin facilita la transición al desarrollo Multiplataforma.

Pilares de la Ingeniería Kotlin

- **Interoperabilidad:** Migración gradual y sin dolor.
- **Concisión:** Menos código, abstracción más alta.
- **Seguridad:** El compilador elimina el NullPointerException.
- **Control:** when como expresión exhaustiva.